Vizard Arctic

C4 Architecture Document

End-to-end система заполнения ледовых пропусков и маршрутизации судов в Арктике

Версия 1.0 | Апрель 2026

Удалённый деплой: Windows 10.78.211.199 | GPU: RTX 2060 SUPER 8GB

Содержание

- 1. System Context — общий контекст системы
- 2. Container View — контейнеры внутри системы
- 3. Backend API — Component View
- 4. Frontend SPA — Component View
- 5. ML Pipeline — Component View
- 6. Reconstruction Flow — динамический процесс
- 7. Route Planning Flow — динамический процесс
- 8. API Contract
- 9. Технологический стек
- 10. Инфраструктура и деплой

1. System Context

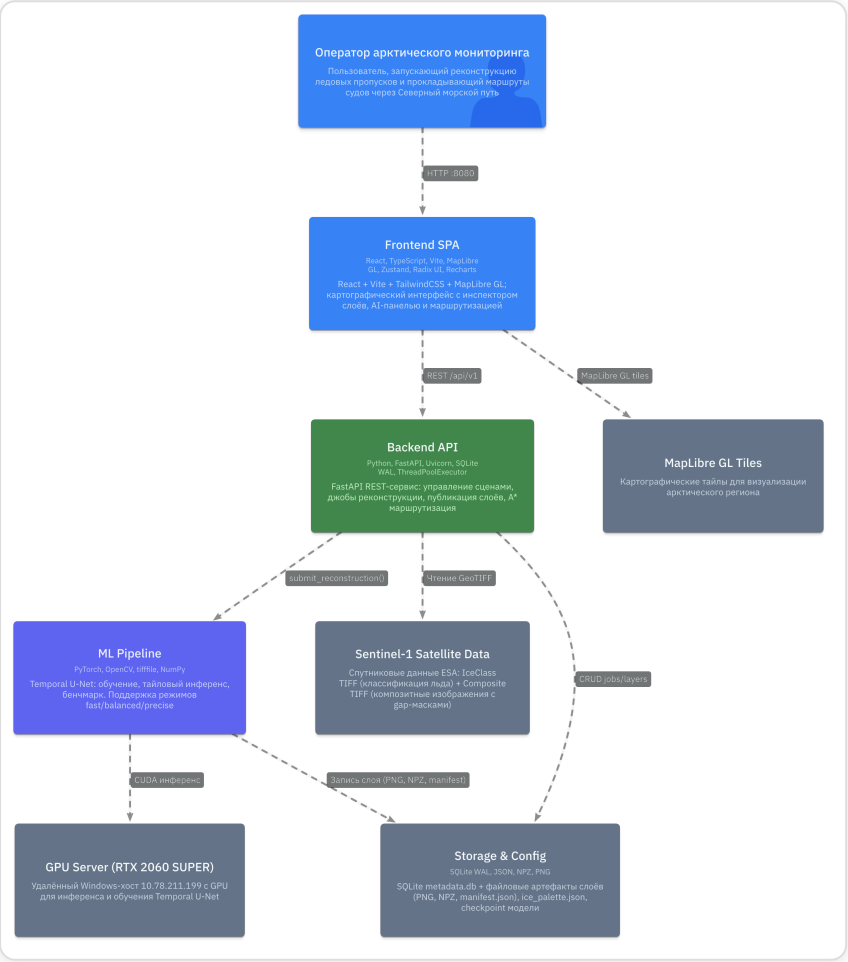
> Vizard Arctic — система, которая читает спутниковые данные Sentinel-1 (IceClass + Composite GeoTIFF), заполняет ледовые пропуски (gap filling) с помощью Temporal U-Net и прокладывает оптимальные маршруты судов через Северный морской путь с учётом ледовых классов и confidence.

Внешние актёры и системы

Элемент	Тип	Описание
Оператор	Person	Пользователь, запускающий реконструкцию и прокладывающий маршруты
Sentinel-1 Data	System	Спутниковые данные ESA: IceClass TIFF + Composite TIFF с gap-масками
GPU Server	System	Windows-хост 10.78.211.199, RTX 2060 SUPER 8GB для U-Net инференса
MapLibre GL Tiles	System	Картографические тайлы для визуализации арктического региона

2. Container View

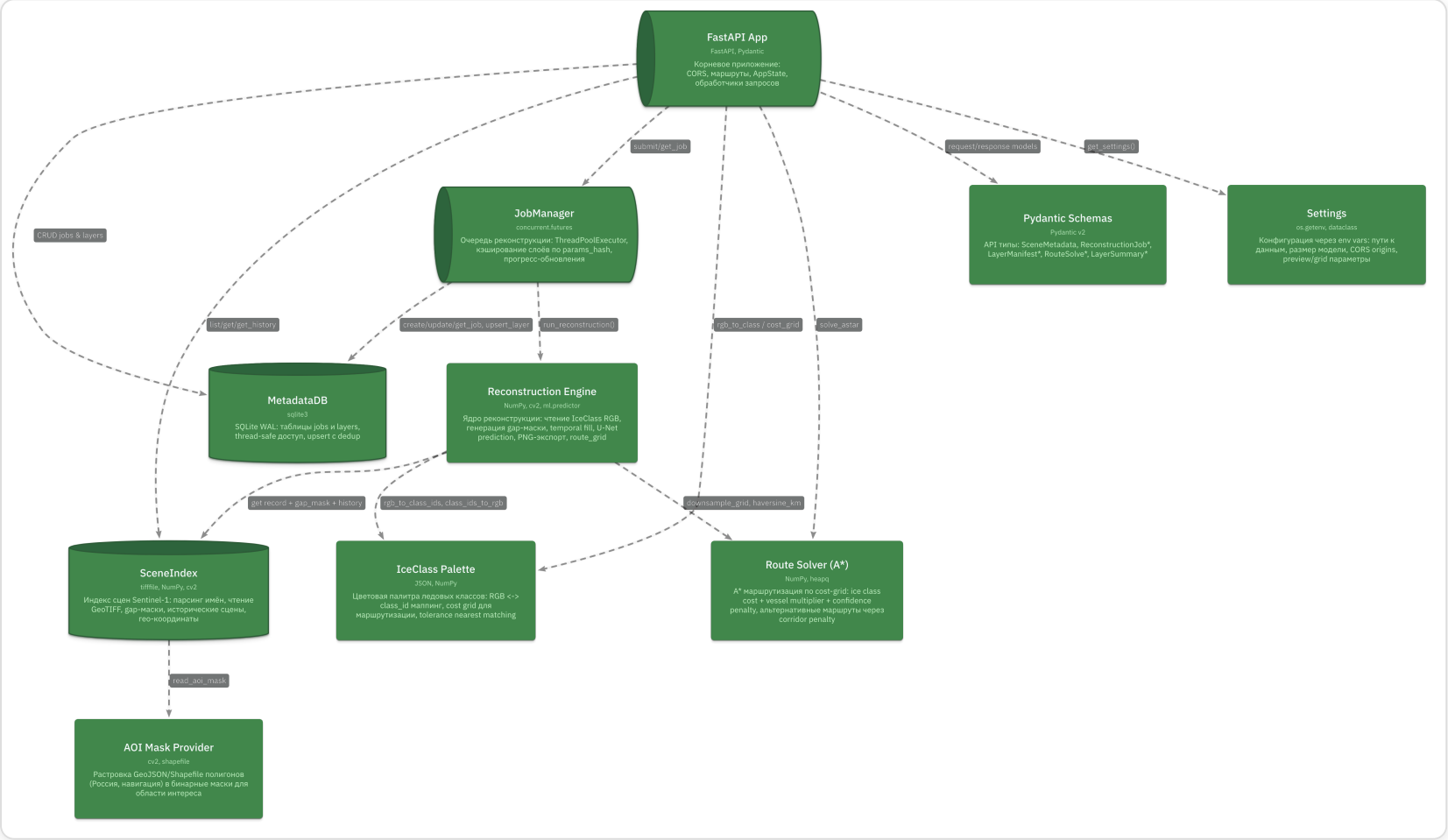
> Четыре контейнера внутри Vizard Arctic: Frontend SPA (React+MapLibre), Backend API (FastAPI+SQLite), ML Pipeline (PyTorch), Storage & Config (файловая система+JSON).



Контейнер	Технология	Описание
Frontend SPA	React Vite MapLibre GL Zustand TailwindCSS	Картографический UI: инспектор слоёв, AI-панель, маршрутизация, справочник судов
Backend API	FastAPI Uvicorn SQLite WAL Pydantic	REST-сервис: управление сценами, джобы реконструкции, публикация слоёв, A* маршрутизация
ML Pipeline	PyTorch OpenCV tifffile NumPy	Temporal U-Net: обучение, тайловый инференс, бенчмарк. Режимы fast/balanced/precise
Storage & Config	SQLite JSON NPZ PNG	metadata.db, слой-артефакты (PNG, NPZ, manifest), ice_palette.json, checkpoint

3. Backend API — Component View

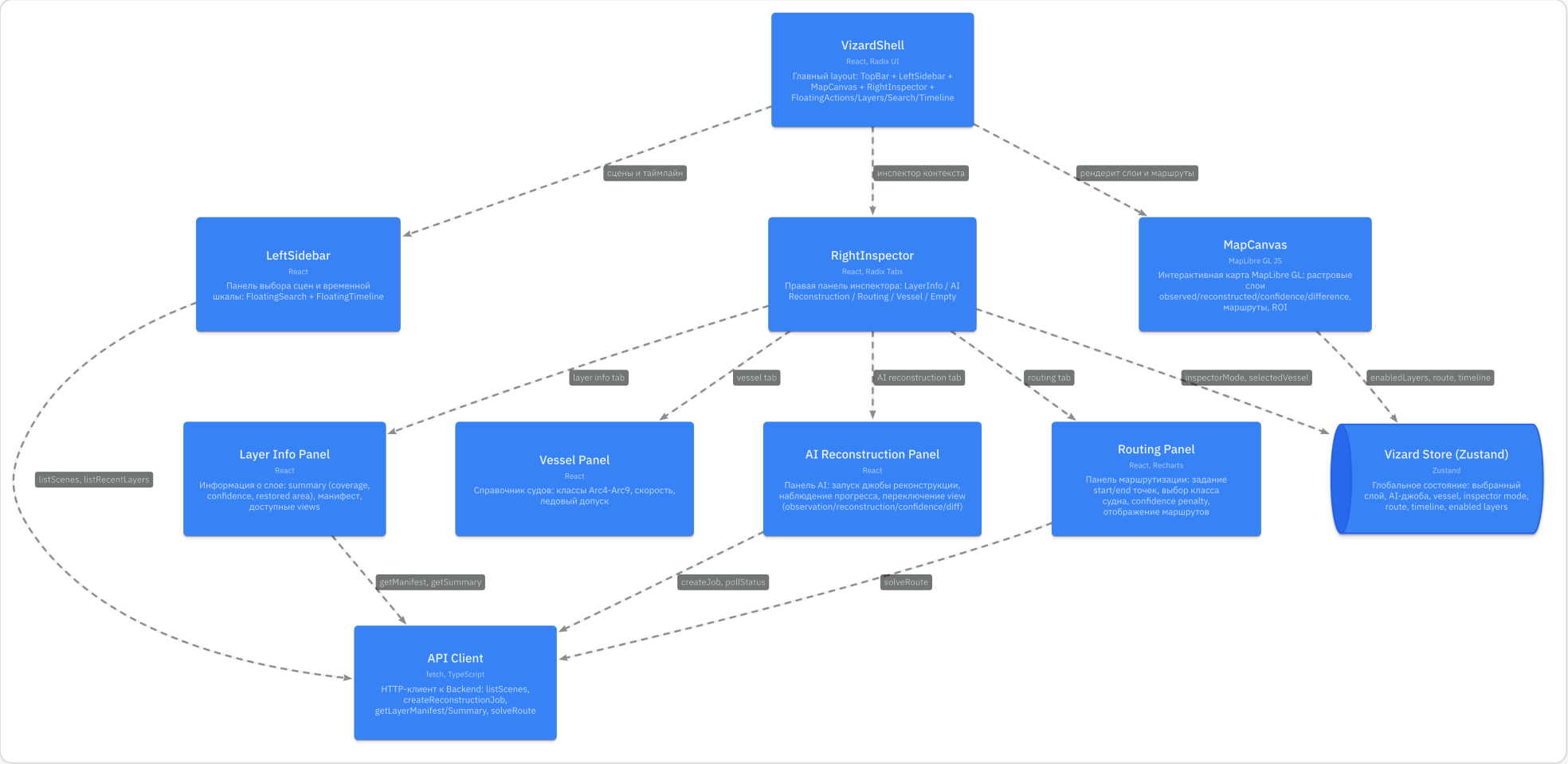
Backend API состоит из 10 компонентов: FastAPI App (точка входа), SceneIndex (индекс сцен), JobManager (очередь джоб), Reconstruction Engine (ядро gap-filling), Route Solver (A*), MetadataDB (SQLite), IceClass Palette (цветовая маппинг), AOI Mask (область интереса), Pydantic Schemas (типы API), Settings (конфигурация).



Компонент	Файл	Описание
FastAPI App	backend/app/main.py	Корневое приложение: CORS, маршруты, AppState, обработчики запросов
SceneIndex	backend/app/scene_index.py	Индекс сцен Sentinel-1: парсинг имён, чтение GeoTIFF, gap-маски, исторические сцены, гео-координаты
JobManager	backend/app/jobs.py	Очередь реконструкции: ThreadPoolExecutor(2), кэширование по params_hash, прогресс-обновления
Reconstruction Engine	backend/app/reconstruction.py	Ядро: чтение IceClass RGB, gap-маска, temporal fill, U-Net prediction, PNG-экспорт, route_grid
Route Solver (A*)	backend/app/route_solver.py	A* маршрутизация: ice class cost + vessel multiplier + confidence penalty, corridor penalty для альтернатив
MetadataDB	backend/app/db.py	SQLite WAL: таблицы jobs/layers, thread-safe, upsert с dedup по (scene_id, params_hash)
IceClass Palette	backend/app/palette.py	RGB <-> class_id маппинг, cost grid, nearest tolerance matching
AOI Mask Provider	backend/app/aoi_mask.py	Растровка GeoJSON/Shapefile полигонов (Россия, навигация) в бинарные маски AOI
Pydantic Schemas	backend/app/schemas.py	API типы: SceneMetadata, ReconstructionJob*, LayerManifest*, RouteSolve*, LayerSummary*
Settings	backend/app/settings.py	Env vars: VIZARD_ICECLASS_DIR, VIZARD_COMPOSITE_DIR, VIZARD_MODEL_CKPT, VIZARD_MODEL_DEVICE, etc.

4. Frontend SPA — Component View

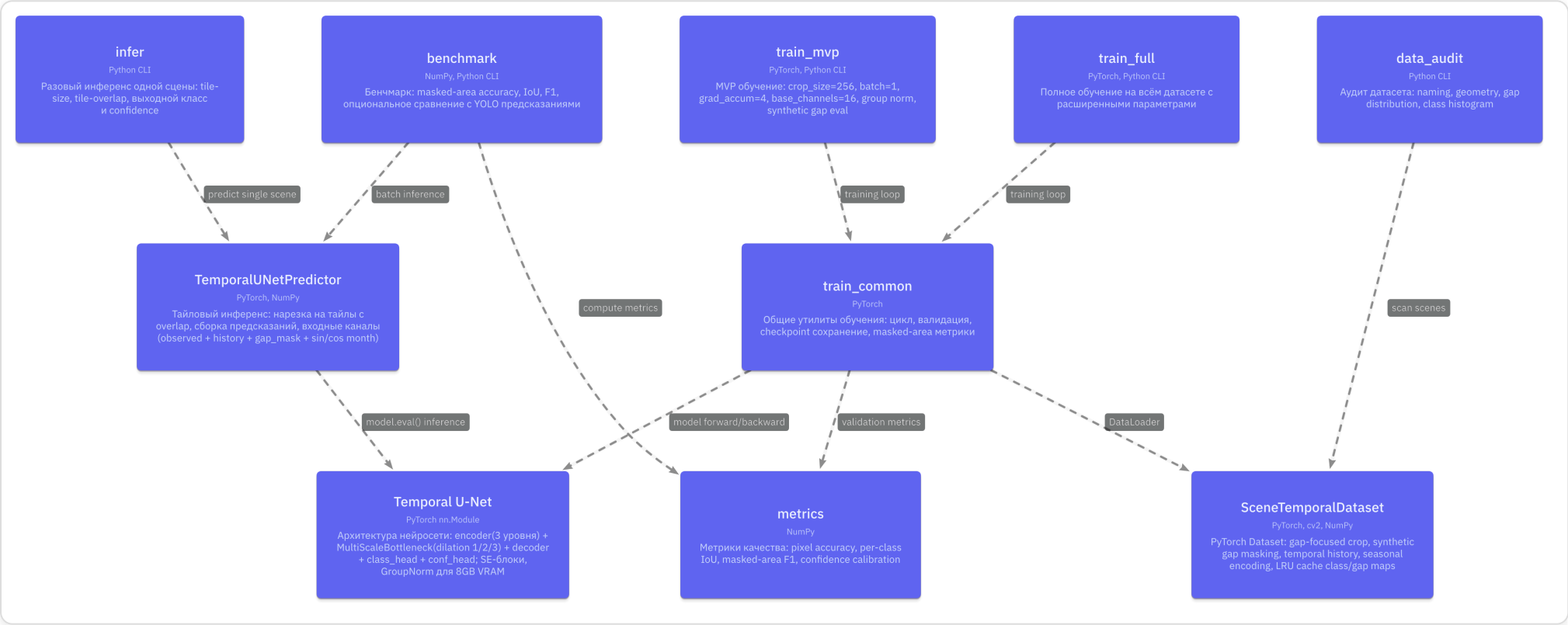
Frontend SPA состоит из 11 компонентов: VizardShell (главный layout), MapCanvas (карта MapLibre), LeftSidebar (сцены + таймлайн), RightInspector (панель контекста с 4 вкладками), AI Reconstruction Panel, Routing Panel, Vessel Panel, Layer Info Panel, Vizard Store (Zustand), API Client.



Компонент	Файл	Описание
VizardShell	components/vizard/VizardShell.tsx	Главный layout: TopBar + LeftSidebar + MapCanvas + RightInspector + Floating panels
MapCanvas	components/vizard/MapCanvas.tsx	Интерактивная карта MapLibre GL: растровые слои, маршруты, ROI
LeftSidebar	components/vizard/LeftSidebar.tsx	Панель сцен: FloatingSearch + FloatingTimeline
RightInspector	components/vizard/RightInspector.tsx	Панель инспектора: LayerInfo / AI Reconstruction / Routing / Vessel / Empty
AI Reconstruction Panel	inspector/AIReconstructionPanel.tsx	Запуск джобы, наблюдение прогресса, переключение view (observation/reconstruction/confidence/diff)
Routing Panel	inspector/RoutingPanel.tsx	Start/end точки, класс судна, confidence penalty, отображение маршрутов
Vessel Panel	inspector/VesselPanel.tsx	Справочник судов: Arc4-Arc9, скорость, ледовый допуск
Layer Info Panel	inspector/LayerInfoPanel.tsx	Summary (coverage, confidence, restored area), манифест, доступные views
Vizard Store	store/vizard-store.ts	Zustand: enabledLayers, selectedLayer, AI job, vessel, inspector mode, route, timeline
API Client	lib/api-client.ts	HTTP-клиент: listScenes, createJob, getManifest/Summary, solveRoute

5. ML Pipeline — Component View

> ML Pipeline состоит из 11 компонентов: Temporal U-Net (архитектура нейросети), TemporalUNetPredictor (тайловый инференс), SceneTemporalDataset (PyTorch Dataset), train_mvp, train_full, train_common (обучение), infer (разовый инференс), benchmark, metrics (оценка качества), data_audit (аудит датасета).



Компонент	Файл	Описание

Temporal U-Net	ml/model.py	Encoder(3 уровня) + MultiScaleBottleneck(dilation 1/2/3) + Decoder + class_head + conf_head; SE-блоки, GroupNorm
TemporalUNetPredictor	ml/predictor.py	Тайловый инференс: нарезка с overlap, сборка, входные каналы (observed + history + gap + sin/cos month)
SceneTemporalDataset	ml/dataset.py	Gap-focused crop, synthetic gap masking, temporal history, seasonal encoding, LRU cache
train_mvp	ml/train_mvp.py	MVP обучение: crop=256, batch=1, grad_accum=4, ch=16, group norm
train_full	ml/train_full.py	Полное обучение на всём датасете
train_common	ml/train_common.py	Общие утилиты: цикл, валидация, checkpoint, masked-area метрики
infer	ml/infer.py	Разовый инференс одной сцены
benchmark	ml/benchmark.py	Masked-area accuracy, IoU, F1, YOLO comparison
metrics	ml/metrics.py	Pixel accuracy, per-class IoU, masked-area F1, confidence calibration
data_audit	ml/data_audit.py	Аудит: naming, geometry, gap distribution, class histogram

Temporal U-Net — архитектура

- **Входные каналы**: observed(1) + history(1-5) + gap_mask(1) + sin_month(1) + cos_month(1) = 4..9
- **Encoder**: 3 уровня ResidualConvBlock (base_ch → 2x → 4x → 8x) с MaxPool2d
- **Bottleneck**: MultiScaleBottleneck — 3 параллельных dilation (1,2,3) + fuse + Dropout2d(0.15)
- **Decoder**: 3 уровня ConvTranspose2d + skip connections + ResidualConvBlock
- **Головы**: class_head (logits, K классов) + conf_head (sigmoid, 0..1)
- **SE-блоки**: Squeeze-and-Excitation (reduction=8) на каждом ResidualConvBlock
- **Нормализация**: GroupNorm (для 8GB VRAM) или BatchNorm2d
- **Инференс**: тайловый (512x512, overlap=64/128), autocrop valid region, model caching

6. Reconstruction Flow

> Процесс AI-реконструкции ледовых пропусков: пользователь выбирает сцену → Frontend отправляет POST запрос → Backend создаёт джоб → JobManager запускает Reconstruction Engine → читает IceClass + Composite → определяет gap-маску → получает исторические сцены → U-Net prediction на GPU → генерирует PNG, NPZ, manifest, summary → сохраняет в Storage → обновляет статус джобы → Frontend опрашивает статус и получает манифест.

Режимы реконструкции

Режим	Модель	Overlap	Описание
fast	Нет (temporal fill)	—	Заполнение из исторических сцен + global mode для остатков; confidence 0.55-0.9
balanced	Temporal U-Net	tile_overlap // 2	Тайловый инференс с умеренным overlap; fallback на temporal fill если модель недоступна

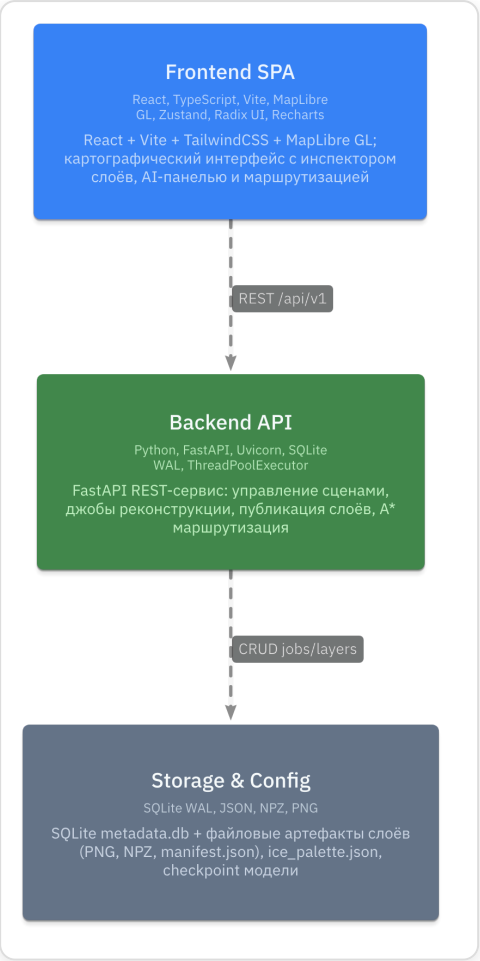
precise	Temporal U-Net	полный tile_overlap	Тайловый инференс с максимальным overlap для лучшего качества

Артефакты слоя (storage/layers/{uuid}/)

Файл	Описание
observed.png	RGBA: исходная IceClass карта с полупрозрачными gap-областями
reconstructed.png	RGBA: восстановленная IceClass карта с подсвеченными gap-областями
confidence.png	RGBA: confidence map (красный→жёлтый→зелёный), полупрозрачные observed области
difference.png	RGBA: diff между observed и reconstructed (красный + фиолетовый)
route_grid.npz	NumPy: classes (uint8), confidence (float32), bounds (float64) — downscaled для A*
manifest.json	layer_id, scene_id, bounds, coordinates, views[]
summary.json	coverage_before/after, restored_area_km2, mean/high/low_confidence, changed_pixels_ratio

7. Route Planning Flow

> Процесс маршрутизации: пользователь задаёт start/end координаты и класс судна → Frontend отправляет POST /api/v1/routes/solve → Backend загружает route_grid.npz из артефактов слоя → строит cost grid из ice class + palette.cost + land_cost → A* поиск оптимального маршрута + 2 альтернативы (через corridor penalty) → возвращает RouteSolveResponse с primary + alternatives (distance, ETA, risk, confidence).



Классы судов и мультипликаторы стоимости

Класс	Мультипликатор	Номинальная скорость (км/ч)
Arc4	2.2	17
Arc5	1.9	18

Arc6	1.55	20
Arc7	1.25	23
Arc9	1.0	26

A* алгоритм

- Cost grid = ice_class_cost (из palette) + land_cost (из NPZ) + confidence_penalty * (1 - confidence)
- Vessel multiplier: Arc4=2.2, Arc9=1.0 — более мощные суда платят меньше за лёд
- 8-connectivity (4 cardinal + 4 diagonal, diagonal weight = sqrt(2))
- Блокировка клеток с cost >= 8000 (непроходимый лёд/земля)
- Альтернативные маршруты: corridor_penalty (radius=5+i, factor=1.55+0.15i) на primary path
- Downsampling: оригинальный raster → 512x512 для скорости A*

8. API Contract

> Backend API доступен на http://127.0.0.1:8000 с префиксом /api/v1. CORS разрешён для localhost:8080 и localhost:5173.

Метод	Endpoint	Описание
GET	/	Информация о сервисе
GET	/health	Health check + количество сцен
GET	/api/v1/scenes	Список сцен Sentinel-1 (limit, offset)
POST	/api/v1/reconstruction/jobs	Создать джобу реконструкции (scene_id, history_steps, model_mode, aoi_bbox)
GET	/api/v1/reconstruction/jobs/{job_id}	Статус джобы (status, progress, layer_id, error)
GET	/api/v1/layers/recent	Список недавних слоёв (limit, offset)
GET	/api/v1/layers/{layer_id}/manifest	Манифест слоя (bounds, coordinates, views[])
GET	/api/v1/layers/{layer_id}/summary	Summary: coverage, confidence, restored_area_km2
GET	/api/v1/layers/{layer_id}/{view}.png	Изображение слоя (observed, reconstructed, confidence, difference)
POST	/api/v1/routes/solve	Проложить маршрут (layer_id, start/end lon/lat, vessel_class, confidence_penalty)

9. Технологический стек

Backend

Технология	Версия	Назначение
Python	3.11/3.14	Основной язык
FastAPI	—	REST API framework
Uvicorn	—	ASGI сервер
Pydantic v2	—	Валидация и сериализация
SQLite (WAL)	—	Метаданные (jobs, layers)
NumPy	—	Матричные операции, A* grid
OpenCV (cv2)	—	Чтение/запись TIFF/PNG, resize
tifffile	—	Чтение GeoTIFF метаданных и растров

Frontend

Технология	Назначение
React 18	UI framework
TypeScript	Типизация
Vite	Сборка и dev-сервер
MapLibre GL	Картографическая визуализация

Zustand	Глобальное состояние
TailwindCSS	Стилизация
Radix UI	Headless UI компоненты
Recharts	Графики маршрутов
react-hook-form + zod	Формы и валидация

ML

Технология	Назначение
PyTorch	Фреймворк глубокого обучения
CUDA (RTX 2060 SUPER)	GPU инференс и обучение
Temporal U-Net	Кастомная архитектура (encoder-bottleneck-decoder + dual head)

10. Инфраструктура и деплой

Удалённый хост (Windows)

Параметр	Значение
Хост	10.78.211.199
ОС	Windows 10/11
GPU	NVIDIA RTX 2060 SUPER 8GB
SSH	ssh -i ~/.ssh/id_ed25520 maksi@10.78.211.199
Данные IceClass	C:\Users\maksi\Downloads\vizard_iceclass\Dataset_2025_IceClass
Данные Composite	C:\Users\maksi\Downloads\vizard_composite\Dataset_2025_composite
Проект	C:\Users\maksi\projects\vizard-arctic

Env Variables

Переменная	По умолчанию	Описание
VIZARD_ICECLASS_DIR	C:\Users\maksi\Downloads\vizard_iceclass\Dataset_2025_IceClass	Папка с IceClass GeoTIFF
VIZARD_COMPOSITE_DIR	C:\Users\maksi\Downloads\vizard_composite\Dataset_2025_composite	Папка с Composite GeoTIFF
VIZARD_STORAGE_DIR	{project}/storage	Директория артефактов
VIZARD_SQLITE_PATH	{storage}/metadata.db	Путь к SQLite БД
VIZARD_MODEL_CKPT	{backend}/checkpoints/mvp_unet.pt	Checkpoint модели

VIZARD_MODEL_DEVICE	cuda	Устройство инференса (cuda/cpu)
VIZARD_MODEL_INPUT_SIZE	512	Размер тайла
VIZARD_MODEL_TILE_OVERLAP	64	Overlap тайлов
VIZARD_PREVIEW_MAX_WIDTH	2048	Макс. ширина preview PNG
VIZARD_ROUTE_GRID_SIZE	512	Размер grid для A*
VIZARD_CORS_ORIGINS	localhost:8080,localhost:5173	CORS origins
VITE_API_BASE_URL	http://127.0.0.1:8000	URL Backend для Frontend

Команды запуска

Компонент	Команда
Backend (Windows)	cd backend && ..\.venv\Scripts\python.exe run.py
Frontend (Windows)	cd frontend && npm.cmd run dev -- --host 0.0.0.0 --port 8080
Frontend (macOS)	cd frontend && npm run dev -- --host 0.0.0.0 --port 8080
C4 Architecture (macOS)	likec4 start ./vizard-arctic-c4